

Requirements Engineering with AI

In class 50 minutes · at home about 60 minutes · 1 point, graded 4×0.25 · due before the next practice session

Your assistant is your choice. The notation is not.

Use any AI assistant you can access. Record the tool **and the exact model name** — "ChatGPT" is not a model name — and freeze both for the whole lab. Three prompts answered by three different models is not an experiment. There is no programming language this week: nothing is implemented. What is fixed for everyone is the notation and the file names — PlantUML for the diagram, the four files under requirements/, and the US-nn / AC-nn / UC-nn ID scheme. They are fixed because a checker reads them and because your work has to be comparable with everyone else's.

No Python? Both checkers are a convenience, not the grade. Section 9 of lab-report.md prints the same checks as a list you can walk by hand, and you lose nothing for doing it that way.

1. The scenario — fixed, and the same for everyone

Smart Campus — study room booking. The KBTU library has a set of small study rooms that students reserve through a campus web application. Do not invent a different system, and do not let the assistant widen this one.

ACTORS — EXACTLY TWO

Actor	Who it is
Student	A registered KBTU student who reserves rooms for individual or group study.
Administrator	Library staff who keep rooms usable and watch how they are used.

WHAT THE SYSTEM DOES — EXACTLY SIX FUNCTIONS

ID	Use case	Notes
UC-01	View availability	Which rooms are free, and when.
UC-02	Book room	Reserve a free room for a time slot.
UC-03	Cancel booking	Release a reservation the student made.
UC-04	Block or unblock room	Take a room out of service and put it back.
UC-05	Review usage	See how rooms are being used over a period.
UC-06	Send confirmation	The system confirms a booking or a cancellation.

BUSINESS RULES — FOUR, NOT NEGOTIABLE

R	Rule
R1	A booking must start in the future.
R2	A booking lasts at most two hours.
R3	Two bookings for the same room may not overlap.
R4	A blocked room cannot be booked.

OUT OF SCOPE — EXPLICITLY

If any of this appears in your stories, the assistant invented it and your review missed it: payments, fees, fines or penalties · check-in, attendance or QR codes · equipment, cleaning or maintenance requests · SMS, push or reminder notifications of any kind beyond UC-06 · account registration, passwords or authentication · waiting lists and queues · anything about screens, colours, databases or servers.

Two things the scenario deliberately does not settle. Nobody will answer these for you.

1. A booking that ends exactly when another begins — is that an overlap under R3?
2. Is exactly two hours allowed under R2, or must a booking be shorter than two hours?

Decide both, write the decision into your assumptions, and declare it in submission.yml. Either answer is accepted. Not deciding is not.

2. What you hand in

Everything lives in week-03/ on a branch called week-03, in your own repository.

File	What it is
lab-report.md	The worksheet. Do not delete or renumber the headings.
AI_USAGE.md	AI disclosure. Required every week.
submission.yml	The declaration — facts only (see section 5).
requirements/user-stories.md	6–8 stories, IDs US-01..., each with a priority and one assumption.
requirements/acceptance-criteria.md	Three stories × 3–5 Given/When/Then criteria, assumptions first.
requirements/use-cases.puml	The PlantUML source of your diagram.
requirements/traceability.md	The table that ties the three artifacts together.
tests/	Two checkers. Do not edit them — a modified checker is flagged on your report.

3. Parts 1 and 2 — user stories, and the review that counts

Open a fresh chat, paste the scenario from section 1, then this prompt exactly as written. Do not improve it: improving prompts was Week 02's experiment, and this week the generation is deliberately unremarkable, because what is graded is the review.

You are a requirements analyst. For the Smart Campus study room booking system, identify Student and Administrator goals. Write 6 to 8 user stories using: As a [role], I want [goal], so that [reason]. Add a priority and one assumption to each story. Stay within the supplied scenario.

Paste the **original, unedited** output into lab-report.md section 2 before you touch it. That copy is the evidence; without it the review has nothing to be a review of.

Then review every story:

- Does it name a real stakeholder — Student or Administrator, not "System" and not "User"?
- Does it describe one valuable outcome, and does the reason explain why that outcome matters?
- Can the team test it? Is it small enough for one iteration?
- Did the AI invent something outside the scenario? **Check it against the out-of-scope list.**

Keep 6–8 revised stories with IDs, priorities and assumptions. Every story you deleted, merged or rewrote goes in lab-report.md section 3 with the reason. This is the part that scores.

4. Part 3 — acceptance criteria

Select three stories. Fresh chat, verbatim prompt:

For each selected story, write 3 to 5 acceptance criteria in Given, When, Then form. Include successful behavior, validation, and an error or alternative case. Apply these rules: bookings must be in the future, maximum duration is two hours, rooms cannot overlap, and blocked rooms cannot be booked. List assumptions before the criteria.

Review each criterion: the starting condition is clear; one action triggers one observable result that could become a software test; terms and business rules stay consistent with section 1; boundary and invalid cases are covered — three happy paths are not a set of criteria; and nothing describes screens, databases or internal classes.

Your assumptions must settle the two open questions from section 1.

5. Parts 4 and 5 — the diagram, traceability, and the checkers

Fresh chat, verbatim prompt:

```
Create PlantUML code for a UML use-case diagram of the Smart Campus study room booking system. Place Student and Administrator outside the system boundary. Include View availability, Book room, Cancel booking, Block or unblock room, Review usage, and Send confirmation. Show only justified actor associations. Use include or extend only when the relationship is clear. Do not model screens, databases, or internal classes.
```

Before it goes in the file: both actors outside the boundary and no third actor; every name a verb plus a business goal; associations that match responsibilities — look hard at who is connected to Review usage and to Send confirmation, and ask whether a person triggers that function at all; no screens or databases; every element traceable to a revised story.

Then fill requirements/traceability.md — one row per use case, listing the stories behind it and the criteria that test it.

Expect gaps. The six use cases are fixed; your stories are not. A use case with nothing behind it is a finding, not a failure — write it down rather than inventing a story to paper over it.

Run both checkers from inside week-03/:

```
python tests/check_requirements.py    # 23 structural checks
python tests/validate_submission.py    # checks your declaration parses
```

6. The declaration — submission.yml

One short file of facts: who you are, which assistant and which exact model, what your checker run returned and at which commit, your decision on the two open questions, your traceability gaps, and three findings. Explanations stay in lab-report.md — nothing is written twice. Fill it last, once your checker run is final.

Two things that decide marks

Your checker numbers are re-run. check_requirements.py is run again at your commit. A FAIL you report and explain costs you nothing. Numbers that do not match the re-run cost the whole workflow criterion.

A finding that names no ID is not a finding. review_findings needs three or more lines, each pointing at a US-nn, AC-nn or UC-nn. "The AI made mistakes" is rejected by the validator, not by me.

7. Submitting

```
git checkout main && git pull
git checkout -b week-03
# work inside week-03/ only
git add week-03
git commit -m "week-03: <what this commit actually does>"
git push -u origin week-03
```

Open a pull request week-03 → main in your own repository and leave it open. Paste the **PR link** into the Teams assignment — not the repository link, not a merged PR, and not email. At least three meaningful commits, all authored by your own GitHub account. The full workflow rules are in SETUP.md and have not changed.

8. Grading — 1 point, 4 × 0.25

Criterion	Full credit	Half credit	No credit
Generation evidence	Three prompts verbatim, three original outputs pasted unedited, tool and exact model named	An output missing or cleaned up before pasting	No original outputs, or the prompts were rewritten
Review quality	Every change recorded with a reason; scope creep caught against the out-of-scope list; at least one wrong association identified	Changes recorded but the reasons are generic	The AI's output was kept as it came

Criterion	Full credit	Half credit	No credit
Requirements artifacts	Four files, consistent; 6–8 stories with priorities and assumptions; three sets of 3–5 criteria including invalid cases; a valid diagram; a traceability table whose gaps are named	One artifact incomplete or inconsistent	Missing artifacts, or IDs that do not resolve
Workflow & disclosure	week-03 branch, ≥3 commits authored by you, open PR with the standard description, AI_USAGE.md complete, submission.yml filled and validating	One element missing, or the declaration incomplete	Committed to main, PR merged, no disclosure, or declared checker numbers that do not match the re-run

The line that matters most this week: a FAIL you report and explain costs you nothing; one you hide costs the whole criterion. The same goes for a gap in the traceability table.

Your assistant, your model and your working style are never part of the mark.

AI policy for this lab: Level D. AI use is permitted and expected where the task says so; you remain responsible for testing, verifying and defending everything you submit, and all AI use must be disclosed in AI_USAGE.md. Undisclosed AI use is an academic-integrity violation, not a style issue.